

SEQUENCE COMPARISONS

When comparing two objects of the built-in sequence types, such as strings, lists, or tuples, Python puts more significance on the earlier items in the sequence. That is, it won't compare the later items unless the earlier items have equal values. For example, enter the following into the interactive shell:

```
>>> 'Azriel' < 'Zelda'
True
>>> (1, 2, 3) > (0, 8888, 9999)
True
```

The string 'Azriel' comes before (in other words, is less than) 'Zelda' because 'A' comes before 'Z'. The tuple (1, 2, 3) comes after (in other words, is greater than) (0, 8888, 9999) because 1 is greater than 0. On the other hand, enter the following into the interactive shell:

```
>>> 'Azriel' < 'Aaron'
False
>>> (1, 0, 0) > (1, 0, 9999)
False
```

The string 'Azriel' doesn't come before 'Aaron' because even though the 'A' in 'Azriel' is equal to the 'A' in 'Aaron', the subsequent 'z' in 'Azriel' doesn't come before the 'a' in 'Aaron'. The same applies to the tuples (1, 0, 0) and (1, 0, 9999): the first two items in each tuple are equal, so it's the third items (0 and 9999, respectively) that determine that (1, 0, 0) comes before (1, 0, 9999).

This forces us to make a design decision about our `WizCoin` class. Should `WizCoin(0, 0, 9999)` come before or after `WizCoin(1, 0, 0)`? If the number of galleons is more significant than the number of sickles or knuts, `WizCoin(0, 0, 9999)` should come before `WizCoin(1, 0, 0)`. Or if we compare objects based on their values in knuts, `WizCoin(0, 0, 9999)` (worth 9,999 knuts) comes after `WizCoin(1, 0, 0)` (worth 493 knuts). In `wizcoin.py`, I decided to use the object's value in knuts because it makes the behavior consistent with how `WizCoin` objects compare with integers and floats. These are the kinds of decisions you'll have to make when designing your own classes.

There are no *reflected* comparison dunder methods, such as `__req__()` or `__rne__()`, that you'll need to implement. Instead, `__lt__()` and `__gt__()` reflect each other, `__le__()` and `__ge__()` reflect each other, and `__eq__()` and `__ne__()` reflect themselves. The reason is that the following relationships hold true no matter what the values on the left or right side of the operator are:

- `purse > [2, 5, 10]` is the same as `[2, 5, 10] < purse`
- `purse >= [2, 5, 10]` is the same as `[2, 5, 10] <= purse`